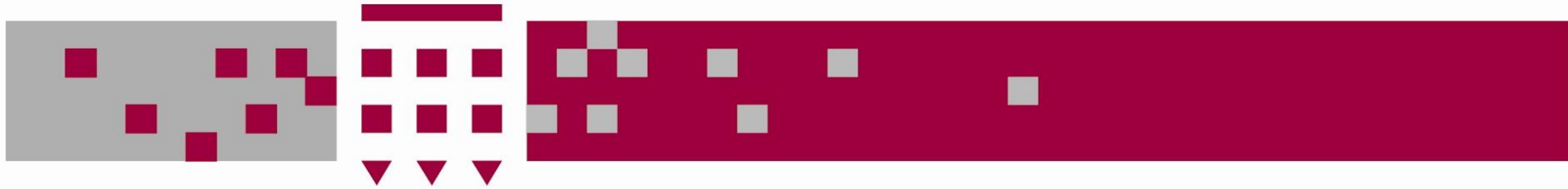


UNIVERSITY OF WESTMINSTER



5COSC019W – Object Oriented Programming Week 11

Dr. Barbara Villarini

b.villarini@westminster.ac.uk

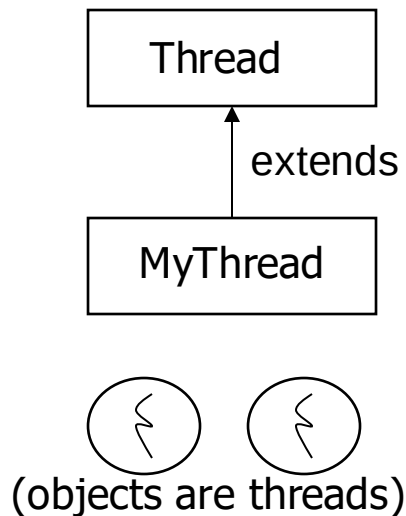


Overview

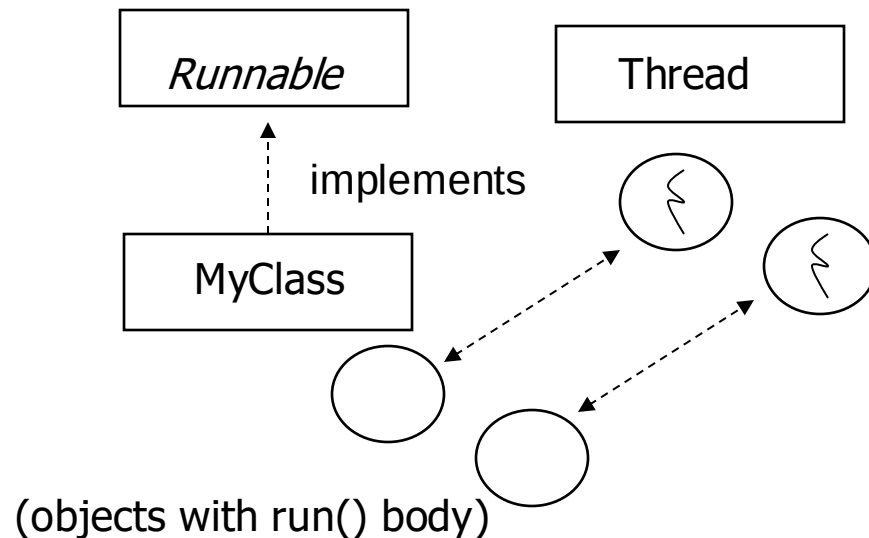
- Recap on Multithreading
- Concurrency
- Synchronization

2 Threading Mechanisms

1. Create a class that extends the `java.lang.Thread` class
2. Create a class that implements the `java.lang.Runnable` interface



[1]



[2]



1st Method: Extending Thread Class

- Create a class by extending Thread class and override **run() method**:

```
class MyThread extends Thread
{
    public void run()
    {
        // thread body of execution
    }
}
```

- Create a thread:

```
MyThread thr1 = new MyThread();
```

- Start Execution of threads:

```
thr1.start();
```



2nd method: Threads by implementing Runnable interface

- Create a class that implements the interface Runnable and override run() method:

```
class MyThread implements Runnable
{
    .....
    public void run()
    {
        // thread body of execution
    }
}
```

- Creating Object:

```
MyThread myObject = new MyThread();
```

- Creating Thread Object:

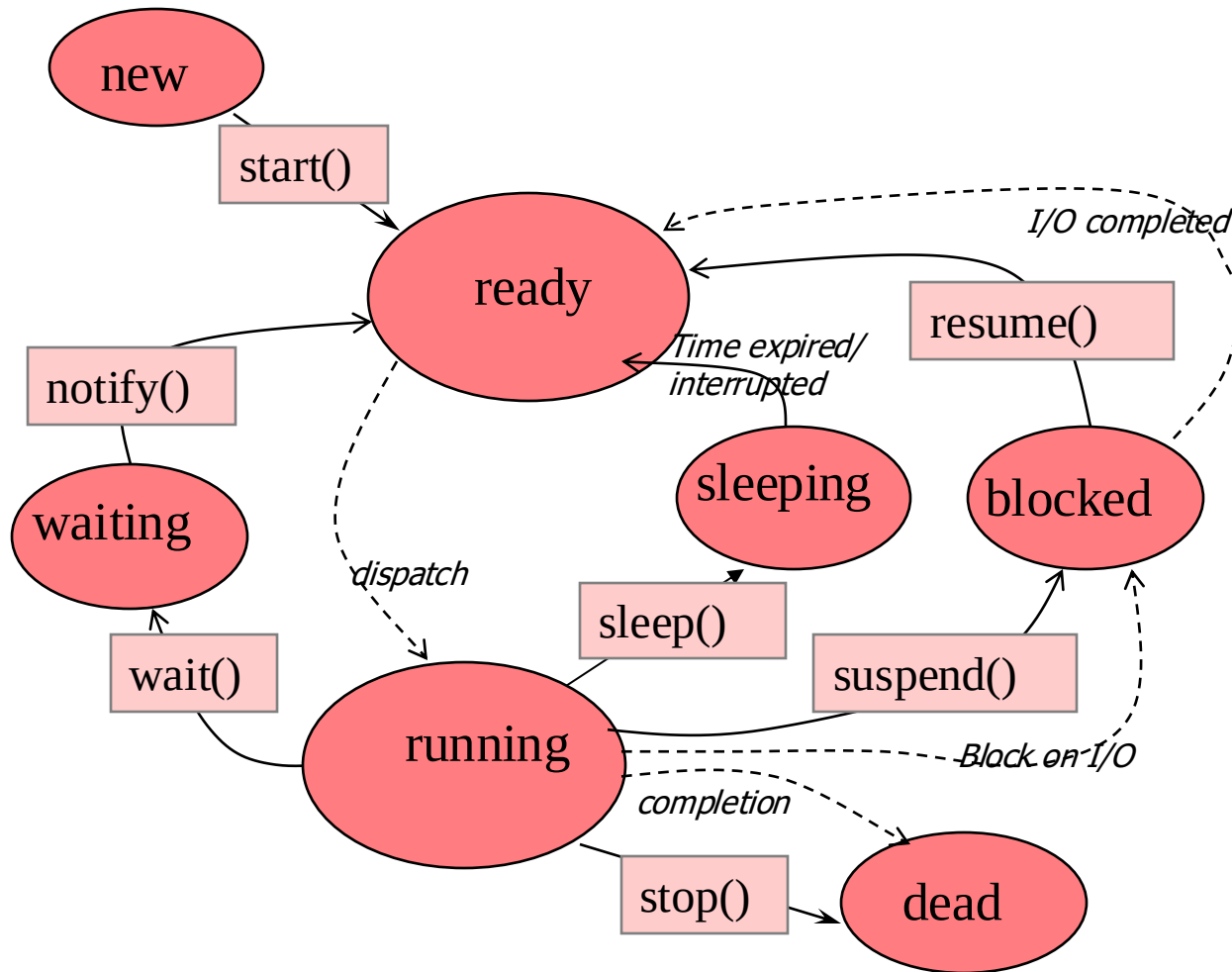
```
Thread thr1 = new Thread( myObject );
```

- Start Execution:

```
thr1.start();
```



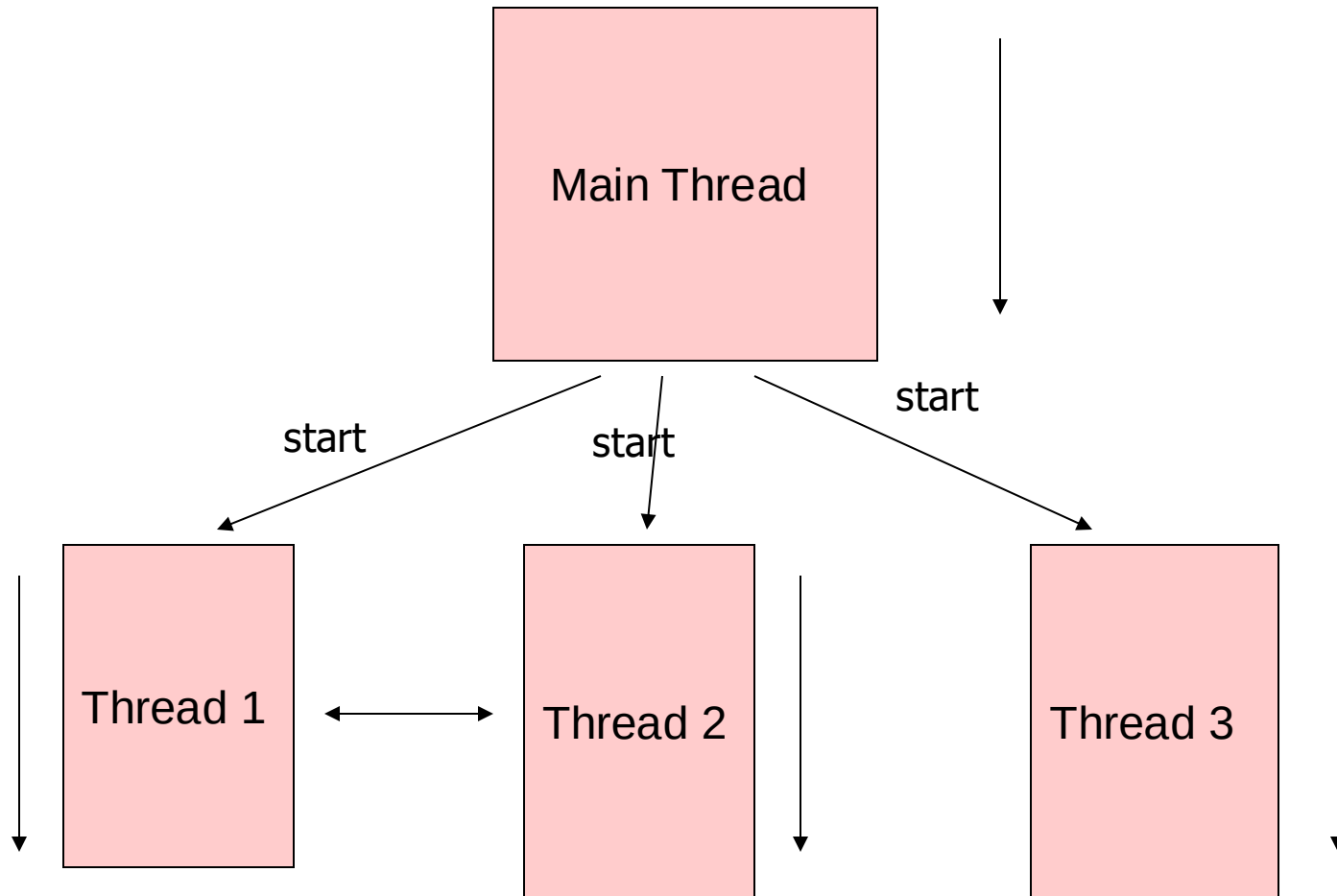
Life Cycle of Thread



A multithread program



threads are light-weight processes within a process



Threads may switch or exchange data/results



Concurrent Access to Data

- Concurrent access to data can lead to data integrity problems
 - Specifically, if two sources attempt to update the same data at the same time, the result of the data can be undefined.
 - The outcome is determined by how the scheduler schedules the two sources.
 - Since the schedulers activities cannot be predicted, the outcome cannot be predicted
- Databases deal with this mechanism through "locking"
 - If a source is going to update a table or record, it can lock the table or record until such time that the data has been successfully updated.
 - While locked, all access is blocked except to the source which holds the lock.
- Java has the equivalent mechanism. It is called **synchronization**

Read/Write problem

- If one thread tries to read the data and another thread tries to update the same data leads to inconsistent state for the shared data.
- This can be prevented by synchronizing access to the data via Java **synchronized** keyword

```
public synchronized void update() {  
    . . .  
}
```

Read/Write Problem example

- Consider an example of a bank offering online access to its customers to perform transactions on their accounts

```
public class InternetBankingSystem {  
    public static void main(String [] args ) {  
        Account accountObject = new Account(100);  
        new Thread(new DepositThread(accountObject, 30)).start();  
        new Thread(new DepositThread(accountObject, 20)).start();  
        new Thread(new DepositThread(accountObject, 10)).start();  
        new Thread(new WithdrawThread(accountObject, 30)).start();  
        new Thread(new WithdrawThread(accountObject, 50)).start();  
        new Thread(new WithdrawThread(accountObject, 20)).start();  
    } // end main()  
}
```



Example – withdraw thread

```
public class WithdrawThread implements Runnable {  
  
    private Account account;  
    private double amount;  
  
    public WithdrawThread(Account account, double amount) {  
        this.account = account;  
        this.amount = amount;  
    }  
  
    public void run() {  
        //make a withdraw  
        account.withdraw(amount);  
    }  
  
} //end WithdrawThread class
```

Example – Deposit thread

```
public class DepositThread implements Runnable {  
  
    private Account account;  
    private double amount;  
  
    public DepositThread(Account account, double amount) {  
        this.account = account;  
        this.amount = amount;  
    }  
  
    public void run() {  
        //make a deposit  
        account.deposit(amount);  
    }  
  
} //end DepositThread class
```



Example – Class Account

```
public class Account {
    private double balance = 0;

    public Account(double balance) {
        this.balance = balance;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            System.out.println("Can't deposit.");
        }
        else{ this.balance += amount;
            System.out.println("Deposit" + amount + " in thread"
                + Thread.currentThread().getId() + ", balance is " + balance);
        }
    }

    public void withdraw(double amount) {
        if (amount < 0 || amount > this.balance) {
            System.out.println ("Can't withdraw.");
        }
        else{this.balance -= amount;
            System.out.println("Withdraw" + amount + " in thread "
                + Thread.currentThread().getId() + ", balance is " +
balance);
        }
    }
}

//end Account class
```



Output

- Without synchronization an unpredictable behavior will occur.
- Run may generate the following printout in the system console:

```
Withdraw 30.0 in thread 11, balance is 60.0  
Deposit 20.0 in thread 9, balance is 60.0  
Withdraw 50.0 in thread 12, balance is 60.0  
Deposit 10.0 in thread 10, balance is 60.0  
Withdraw 20.0 in thread 13, balance is 60.0  
Deposit 30.0 in thread 8, balance is 60.0
```

How we solve this problem?

- How will we ensure that only one class can access the data at one time ?
- We can use the **Synchronized** keyword

Example

```
public class Account {
    private double balance = 0;

    public Account(double balance) {
        this.balance = balance;
    }

    public synchronized void deposit(double amount) {
        if (amount < 0) {
            System.out.println ("Can't deposit.");
        }
        this.balance += amount;
        System.out.println("Deposit" + amount + " in thread"
            + Thread.currentThread().getId() + ", balance is " + balance);
    }

    public synchronized void withdraw(double amount) {
        if (amount < 0 || amount > this.balance) {
            System.out.println ("Can't withdraw.");
        }
        this.balance -= amount;
        System.out.println("Withdraw" + amount + " in thread "
            + Thread.currentThread().getId() + ", balance is " +
balance);
    }
}

} //end Account class
```




Output

- The synchronized method will ensure that only one class can access the data at one time, other operations have to wait until the first operation finishes
- Simply run the InternetBankingSystem program, the following results should be shown on the system console:

```
Deposit 30.0 in thread 8,new balance is 130.0  
Withdraw 50.0 in thread 12,new balance is 80.0  
Deposit 10.0 in thread 10,new balance is 90.0  
Withdraw 20.0 in thread 13,new balance is 70.0  
Withdraw 30.0 in thread 11,new balance is 40.0  
Deposit 20.0 in thread 9,new balance is 60.0
```

Synchronizing Instance Methods and Static Methods

- A synchronized method acquires a lock before it executes.
- **Instance method:** the lock is on the object for which it was invoked.
- **Static method:** the lock is on the class.
- If one thread invokes a synchronized instance method (respectively, static method) on an object, the lock of that object (respectively, class) is acquired, then the method is executed, and finally the lock is released.
- Another thread invoking the same method of that object (respectively, class) is blocked until the lock is released.

Synchronization Using Locks

- A synchronized instance method implicitly acquires a lock on the instance before it executes the method.
- You can use locks explicitly to obtain more control for coordinating threads.
- A lock is an instance of the **Lock** interface, which declares the methods for acquiring and releasing locks.



Lock implementation

- **ReentrantLock**: concrete implementation of Lock for creating mutually exclusive locks.

```
Lock balanceLock = new ReentrantLock()
```

```
...
```

```
balanceLock.lock();
```

```
Manipulate the shared code here
```

```
balanceLock.unlock();
```

```
...
```

Example using locks



```
public class Account {  
    private double balance = 0;  
    private Lock balanceLock;
```

Declare the lock

```
    public Account(double balance) {  
        this.balance = balance;
```

```
    balanceLock = new ReentrantLock();
```

Instantiate the lock

```
}
```

```
public void deposit(double amount) {  
    if (amount < 0) {  
        System.out.println ("Can't deposit.");  
    }
```

```
    balanceLock.lock();
```

```
    this.balance += amount;
```

```
    System.out.println("Deposit" + amount + " in thread "  
        + Thread.currentThread().getId() + ", balance is " + balance);
```

```
    balanceLock.unlock();
```

```
}
```

```
public void withdraw(double amount) {  
    if (amount < 0 || amount > this.balance) {  
        throw new IllegalArgumentException("Can't withdraw.");  
    }
```

```
    balanceLock.lock();
```

```
    this.balance -= amount;
```

```
    System.out.println("Withdraw" + amount + " in thread "  
        + Thread.currentThread().getId() + ", balance is " + balance);
```

```
    balanceLock.unlock();
```

```
}
```

```
}//end Account class
```



Lock
the
access
to
balance





Synchronizing Statements

- A *synchronized block* can be used to acquire a lock on any object, when executing a block of code.

```
synchronized (expr) {  
    statements;  
}
```

- `expr` must evaluate to an object reference.
- If the object is already locked by another thread, the thread is blocked until the lock is released.
- When a lock is obtained on the object, the statements in the synchronized block are executed, and then the lock is released.



wait(), notify(), and notifyAll()

- Use the `wait()`, `notify()`, and `notifyAll()` methods to facilitate communication among threads.
- *They must be called in a synchronized method or a synchronized block on the calling object of these methods. Otherwise, an IllegalMonitorStateException would occur.*
- The `wait()` method lets the thread wait until some condition occurs. When it occurs, you can use the `notify()` or `notifyAll()` methods to notify the waiting threads to resume normal execution.
- The `notifyAll()` method wakes up all waiting threads, while `notify()` picks up only one thread from a waiting queue.



wait() and notify()

- When **wait()** is invoked, it pauses the thread and simultaneously releases the lock on the object.
- When the thread is restarted **after being notified**, the lock is automatically reacquired.

Example wait(),notify()



```
public class Myclass {  
    public static void main(String[] args){  
        ThreadB b = new ThreadB();  
        b.start();  
  
        synchronized(b) {  
            try{  
                System.out.println("Waiting for b to complete...");  
                b.wait();  
            }catch(InterruptedException e){  
                e.printStackTrace();  
            }  
  
            System.out.println("Total is: " + b.total);  
        }  
    }  
}
```

```
class ThreadB extends Thread{  
    int total;  
    @Override  
    public void run(){  
        synchronized(this){  
            for(int i=0; i<100 ; i++){  
                total += i;  
            }  
            notify();  
        }  
    }  
}
```

Output:

Waiting for b to complete...

Total is: 4950

Synchronization Mechanism

- The main thread (MyClass) starts ThreadB and then enters a synchronized block waiting for ThreadB to notify it.
 - ThreadB calculates the sum in the run() method and then calls notify() to signal the main thread to continue.
-
- **Flow of Execution:**
 1. The main thread creates and starts ThreadB.
 2. The main thread enters the synchronized block on ThreadB and waits for it to complete by calling `b.wait()`.
 3. ThreadB runs and calculates the total ($0 + 1 + 2 + \dots + 99$).
 4. When ThreadB finishes, it calls `notify()` to notify the main thread that it can proceed.
 5. The main thread resumes, prints the total, and ends.

Suspending and resuming threads

- You need two flag variables:
 - One for suspend and resume
 - One for stop

```
boolean suspended;  
boolean stopped;
```

- If `suspended == true` then the thread must pause
- If `suspended == false` then the thread must continue
- If `stopped == true` then the thread must terminate

Example



We implement a thread that counts numbers and it will be suspended, resumed and stopped from the main program

```
public class MyThread implements Runnable{
```

```
    boolean stopped;  
    boolean suspended;
```

} Flags

```
    //constructor
```

```
    public MyThread(){  
        suspended = false;  
        stopped = false;  
    }
```

```
    // the entry point for the thread
```

```
    public void run(){  
        System.out.println("My thread starting.");  
        try{  
            for(int i = 0; i < 1000; i++){  
                System.out.println(i);  
                Thread.sleep(500);
```

} Print numbers

```
                // Use synchronized block to check suspended and stopped  
                synchronized(this){  
                    while(suspended)  
                        wait();
```

← If it is suspended then wait...

```
                }  
                if (stopped)  
                    break;
```

← If it is stopped then exit...

```
            }
```

```
    }
```

...continue

```
catch (InterruptedException e) {
    System.out.println("My thread is interrupted");
}
System.out.println("My thread exiting.");
}

//stop the thread
→ synchronized void mystop() {
    stopped = true;
    // the following ensure that a thread that was suspended is stopped
    suspended = false;
    notify();
}

//suspended the thread
→ synchronized void mysuspend() {
    suspended = true;
}

// resume
→ synchronized void myresume() {
    suspended = false;
    notify();
}
}
```



```
public static void main(String[] args) {  
    MyThread mythread = new MyThread();  
  
    Thread t = new Thread(mythread);  
    t.start();  
  
    try{  
        Thread.sleep(1000); // t is running  
  
        mythread.mysuspend(); // suspend the thread  
        System.out.println("Suspending thread");  
        Thread.sleep(1000);  
  
        mythread.myresume(); // resume the thread  
        System.out.println("Resuming thread");  
        Thread.sleep(1000);  
  
        mythread.mysuspend(); // suspend again the thread  
        System.out.println("Suspending thread");  
        Thread.sleep(1000);  
  
        mythread.myresume(); // resume the thread  
        System.out.println("Resuming thread");  
        Thread.sleep(1000);  
  
        //stopping thread  
        mythread.mystop();  
    }  
    catch(InterruptedException e){  
        System.out.println("Main thread interrupted");  
    }  
  
    System.out.println("Main thread exiting");  
}
```

Key Concepts Demonstrated in the Code

- **Thread Synchronization:** Using synchronized blocks to manage shared resources (stopped and suspended flags).
- **Thread Communication:** Using wait() and notify() for controlling the execution of threads.
- **Thread States:** The thread can be in **running**, **suspended**, or **stopped** states depending on the method invoked.